Prathiksha Patil
261100690008

## Lab 3:

## Cryptology Lab — Introduction to Python

## Python Basics and Debugging

1. Debug the following programs. Identify the type of error in each case.

print("Welcome to Cryptology Lab)
message = "Python Programming"
print(mesage)
num = input("Enter a number: ")
result = num + 10
print(result)

```
print("Welcome to Cryptology Lab")
message = "Python Programming"
print(message)
num = int(input("Enter a number: "))
result = num + 10
print(result)
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q1.py
Welcome to Cryptology Lab
Python Programming
Enter a number: 5
15
```

2. Write Python statements to display the following exactly:

Welcome to Cryptology Lab
It's interesting to learn Python.
Students, "Welcome to the Cryptology Lab".

```
print("Welcome to Cryptology Lab")
print("It's interesting to learn python")
print('Students, "Welcome to Cryptology Lab".')
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q2.py
Welcome to Cryptology Lab
It's interesting to learn python
Students, "Welcome to Cryptology Lab".
```

3. Read two numbers from the user and perform:

Addition
Subtraction
Multiplication
Division
Integer division
Remainder
Exponentiation

Display each result with an appropriate message.

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Addition =", num1 + num2)
print("Subtraction =", num1 - num2)
print("Multiplication =", num1 * num2)
print("Division =", num1 / num2)
print("Integer division =", num1 // num2)
print("Remainder =", num1 %  num2)
print("Exponentiation =", num1 ** num2)
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q3.py
Enter first number: 2
Enter second number: 3
Addition = 5.0
Subtraction = -1.0
Multiplication = 6.0
Division = 0.6666666666666666
Integer division = 0.0
Remainder = 2.0
Exponentiation = 8.0
```

4. Read a floating-point number and display:

its data type;
its absolute value;
the value rounded to two decimal places.
Part B — Data Types and Collections (~25 min)

```python
num = float(input("Enter a floating-point number: "))
print("Data type:", type(num))
print("Absolute value:", abs(num))
print("Rounded value:", round(num, 2))
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q4.py
Enter a floating-point number: -25.6789
Data type: <class 'float'>
Absolute value: 25.6789
Rounded value: -25.68
```

5. Create variables containing examples of:

int, float, complex, str, list, tuple, set, and dictionary.

Use Python to display the value and data type of each variable.

```python
a = 10
b = 10.5
c = 3 + 4j
d = "Cryptology"
e = [1, 2, 3, 4, 5]
f = (10, 20, 30)
g = {1, 2, 3, 4}
h = {"name": "Student", "age": 21}
print("Value:", a, "Data type:", type(a))
print("Value:", b, "Data type:", type(b))
print("Value:", c, "Data type:", type(c))
print("Value:", d, "Data type:", type(d))
print("Value:", e, "Data type:", type(e))
print("Value:", f, "Data type:", type(f))
print("Value:", g, "Data type:", type(g))
print("Value:", h, "Data type:", type(h))
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q5.py
Value: 10 Data type: <class 'int'>
Value: 10.5 Data type: <class 'float'>
Value: (3+4j) Data type: <class 'complex'>
Value: Cryptology Data type: <class 'str'>
Value: [1, 2, 3, 4, 5] Data type: <class 'list'>
Value: (10, 20, 30) Data type: <class 'tuple'>
Value: {1, 2, 3, 4} Data type: <class 'set'>
Value: {'name': 'Student', 'age': 21} Data type: <class 'dict'>
```

6. Demonstrate experimentally that:

Strings are immutable, whereas lists are mutable.

Include the Python statements you used and explain the observed result.

```python
string = "HELLO"
print("Original string:", string)

try:
    string[0] = "Y"
except TypeError:
    print("String cannot be changed because strings are immutable.")

my_list = ["H", "E", "L", "L", "O"]
print("Original list:", my_list)

my_list[0] = "Y"
print("Modified list:", my_list)
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q6.py
Original string: HELLO
String cannot be changed because strings are immutable.
Original list: ['H', 'E', 'L', 'L', 'O']
Modified list: ['Y', 'E', 'L', 'L', 'O']
```

7. Create a list containing five prime numbers. Perform the following:

display the first three elements;
display the last element;
append another prime number;
extend the list with two more prime numbers;
display the total number of elements.

Part C — String Operations

For Questions 8–12, use:

message = "CRYPTOLOGY USING PYTHON"

```python
prime = [2, 3, 5, 7, 11]

print("First three elements:", prime[:3])
print("Last element:", prime[-1])

prime.append(13)
print("After appending:", prime)

prime.extend([17, 19])
print("After extending:", prime)

print("Total number of elements:", len(prime))
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q7.py
First three elements: [2, 3, 5]
Last element: 11
After appending: [2, 3, 5, 7, 11, 13]
After extending: [2, 3, 5, 7, 11, 13, 17, 19]
Total number of elements: 8
```

8. Display:

length of the string;
first five characters;
last six characters;
characters from positions 3 to 8;
string in reverse order.

```python
message = "CRYPTOLOGY USING PYTHON"

print("Length of the string:", len(message))
print("First five characters:", message[:5])
print("Last six characters:", message[-6:])
print("Characters from positions 3 to 8:", message[3:9])
print("String in reverse order:", message[::-1])
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q8.py
Length of the string: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from positions 3 to 8: PTOLOG
String in reverse order: NOHTYP GNISU YGOLOTPYRC
```

9. Perform the following:

convert to lowercase;
convert back to uppercase;
check whether "PYTHON" exists in the string;
replace "PYTHON" with "PROGRAMMING".

```python
message = "CRYPTOLOGY USING PYTHON"

print("Lowercase:", message.lower())
print("Uppercase:", message.upper())
print("PYTHON exists:", "PYTHON" in message)
print("After replacement:", message.replace("PYTHON", "PROGRAMMING"))
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q9.py
Lowercase: cryptology using python
Uppercase: CRYPTOLOGY USING PYTHON
PYTHON exists: True
After replacement: CRYPTOLOGY USING PROGRAMMING
```

10. Remove the spaces from the string and display the resulting string.

```python
message = "CRYPTOLOGY USING PYTHON"

result = message.replace(" ", "")

print("String without spaces:", result)
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q10.py
String without spaces: CRYPTOLOGYUSINGPYTHON
```

11. Using a loop, display every character and its corresponding Unicode/ASCII value using ord().

Example:

C -> 67
R -> 82
Y -> 89
...

```python
message = "CRYPTOLOGY USING PYTHON"

for character in message:
    print(character, "→", ord(character))
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q11.py
C → 67
R → 82
Y → 89
P → 80
T → 84
O → 79
L → 76
O → 79
G → 71
Y → 89
  → 32
U → 85
S → 83
I → 73
N → 78
G → 71
  → 32
P → 80
Y → 89
T → 84
H → 72
O → 79
N → 78
```

12. Use chr() to convert the following values into characters:

65, 67, 73, 80, 72, 69, 82

Combine the resulting characters and identify the word.

Part D — Text Processing for Cryptology

```
numbers = [65, 67, 73, 80, 72, 69, 82]

word = ""

for number in numbers:
    word = word + chr(number)

print("Characters:", word)
print("Word:", word)
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q12.py
Characters: ACIPHER
Word: ACIPHER
```

13. Plaintext preprocessing

Read a message from the user and:

remove leading/trailing spaces;
convert it to uppercase;
remove spaces between words.

Example:

Input :  meet me tomorrow
Output: MEETMETOMORROW

```
message = input("Enter a message: ")

message = message.strip()
message = message.upper()
message = message.replace(" ", "")

print("Output:", message)
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q13.py
Enter a message: What are you doing
Output: WHATAREYOUDOING
```

14. Character grouping

Read a string and divide it into blocks of five characters.

Example:

Input: CRYPTOLOGYLAB

Output:
CRYPT
OLOGY
LAB

```
message = input("Enter a string: ")

for i in range(0, len(message), 5):
    print(message[i:i+5])
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q14.py
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LAB
```

15. Padding
Modify Question 14 so that an incomplete final block is padded with X.
Input: CRYPTOLOGYLAB
Output:
CRYPT
OLOGY
LABXX

```
message = input("Enter a string: ")

for i in range(0, len(message), 5):
    block = message[i:i+5]

    if len(block) < 5:
        block = block + "X" * (5 - len(block))

    print(block)
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q15.py
Enter a string: CRYPTOLOGYLAB
CRYPT
OLOGY
LABXX
```

16. Character frequency
For the following text:
text = "CRYPTOLOGYISINTERESTING"
write a program to calculate the frequency of each character.
Then identify the most frequently occurring character.

```
text = "CRYPTOLOGYISINTERESTING"

frequency = {}

for character in text:
    if character in frequency:
        frequency[character] = frequency[character] + 1
    else:
        frequency[character] = 1

for character, count in frequency.items():
    print(character, "→", count)

most_frequent = max(frequency, key=frequency.get)

print("Most frequently occurring character:", most_frequent)
print("Frequency:", frequency[most_frequent])
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q16.py
C → 1
R → 2
Y → 2
P → 1
T → 3
O → 2
L → 1
G → 2
I → 3
S → 2
N → 2
E → 2
Most frequently occurring character: T
Frequency: 3
```

17. Percentage frequency
Extend Question 16 to calculate:
Percentage Frequency=
Total Number of Characters/ Frequency of Character ×100
Display the percentage rounded to two decimal places.
Part E — Mathematical Foundations for Cryptology

```
text = "CRYPTOLOGYISINTERESTING"

frequency = {}

for character in text:
    if character in frequency:
        frequency[character] = frequency[character] + 1
    else:
        frequency[character] = 1

total = len(text)

for character, count in frequency.items():
    percentage = (count / total) * 100
    print(character, "→", count, "→", round(percentage, 2), "%")
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q17.py
C → 1 → 4.35 %
R → 2 → 8.7 %
Y → 2 → 8.7 %
P → 1 → 4.35 %
T → 3 → 13.04 %
O → 2 → 8.7 %
L → 1 → 4.35 %
G → 2 → 8.7 %
I → 3 → 13.04 %
S → 2 → 8.7 %
N → 2 → 8.7 %
E → 2 → 8.7 %
```

18. Modular arithmetic
Use Python to evaluate:
29mod26
55mod26
78mod26
−3mod26
−29mod26
Then write a program that accepts an integer and calculates the integer modulo 26.

```
print("29 mod 26 =", 29 % 26)
print("55 mod 26 =", 55 % 26)
print("78 mod 26 =", 78 % 26)
print("-3 mod 26 =", -3 % 26)
print("-29 mod 26 =", -29 % 26)

num = int(input("Enter an integer: "))
print("The number modulo 26 is:", num % 26)
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q18.py
29 mod 26 = 3
55 mod 26 = 3
78 mod 26 = 0
-3 mod 26 = 23
-29 mod 26 = 23
Enter an integer: 3
The number modulo 26 is: 3
```

19. Alphabet representation
Using
A=0, B=1,…,Z=25
write a program that accepts an uppercase character and displays its numerical representation.
Example:
Enter character: M
M -> 12
Also perform the reverse conversion:
Enter number: 12
12 -> M

```
character = input("Enter character: ")

number = ord(character) - ord('A')

print(character, "→", number)

number = int(input("Enter number: "))

character = chr(number + ord('A'))

print(number, "→", character)
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q19.py
Enter character: c
c → 34
Enter number: 22
22 → W
```

20. Prime numbers
Write a program to:
determine whether a given number is prime;
display all prime numbers within a range specified by the user.

```
num = int(input("Enter a number: "))

if num < 2:
    print(num, "is not a prime number")
else:
    prime = True

    for i in range(2, num):
        if num % i == 0:
            prime = False
            break

    if prime:
        print(num, "is a prime number")
    else:
        print(num, "is not a prime number")

start = int(input("Enter starting number: "))
end = int(input("Enter ending number: "))

print("Prime numbers in the range:")

for num in range(start, end + 1):
    if num < 2:
        continue

    prime = True

    for i in range(2, num):
        if num % i == 0:
            prime = False
            break

    if prime:
        print(num, end=" ")
```

```
(prathiksha㉿kali)-[~/crypto_lab]
$ python3 q20.py
Enter a number: 20
20 is not a prime number
Enter starting number: 3
Enter ending number: 7
Prime numbers in the range:
3 5 7
```

21. GCD and coprimes
Read two integers from the user. Find their GCD and determine whether they are coprime.
Test your program with:
15 and 26
18 and 24
17 and 26

```python
import math

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

gcd = math.gcd(num1, num2)

print("GCD =", gcd)

if gcd == 1:
    print("The numbers are coprime.")
else:
    print("The numbers are not coprime.")
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q21.py
Enter first number: 22
Enter second number: 1
GCD = 1
The numbers are coprime.
```

22.
Write a program to get the multiplicative inverse of an integer b.
Part F — Challenge / If Time Permits (~15 min)

```python
import math

b = int(input("Enter an integer b: "))

if math.gcd(b, 26) != 1:
    print("Multiplicative inverse does not exist.")
else:
    for x in range(1, 26):
        if (b * x) % 26 == 1:
            print("Multiplicative inverse of", b, "modulo 26 is:", x)
            break
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 q22.py
Enter an integer b: 5
Multiplicative inverse of 5 modulo 26 is: 21
```

23. WAP in python to implement client and server communication.

Whether you succeed or not is irrelevant - there is no such thing. Making your known is the important thing --- Art and Letters Georgia O'Keeffe.

```python
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind(("127.0.0.1", 5000))
server.listen(1)

print("Server is waiting for connection...")

connection, address = server.accept()

print("Client connected:", address)

message = connection.recv(1024).decode()
print("Message from client:", message)

connection.send("Hello from server".encode())

connection.close()
server.close()
```

```
┌──(prathiksha㉿kali)-[~/crypto_lab]
└─$ python3 server.py
Server is waiting for connection...
Client connected: ('127.0.0.1', 42670)
Message from client: hello world
```

```python
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(("127.0.0.1", 5000))

message = input("Enter message: ")

client.send(message.encode())

response = client.recv(1024).decode()

print("Message from server:", response)

client.close()
```

```
┌──(prathiksha㉿kali)-[~]
└─$ python3 client.py
Enter message: hello world
Message from server: Hello from server
```